

Software & Subscription Plan

Backend,
nutrition, AI,
payments, and
launch
readiness

Recommendation	The proposed stack is enough for the ONE UP MVP and gives the app a professional production path. Start with Supabase for the backbone, then activate paid APIs only when the matching feature is being built or tested.
Current status	UI shell and major screens are complete enough to move into backend planning. The next step is not buying every tool at once. The next step is creating the database, auth rules, and secure API structure.
Cost approach	Subscribe one by one. This avoids paying for Nutritionix, OpenAI usage, RevenueCat, or store submission tools before the app is ready to use them.

1. Recommended Software Stack

Area	Recommended Software	How ONE UP Uses It	Activate When
Main database	Supabase Postgres	User profiles, progress, workouts, nutrition logs, challenges, rewards, subscriptions, and app settings.	Now. This is the app backbone.
Authentication	Supabase Auth	Email/social login, user sessions, protected data, and role-based access with Row Level Security.	Now, with the database setup.
Backend functions	Supabase Edge Functions	Secure server-side logic for API keys, Nutritionix requests, OpenAI meal suggestions, subscription webhooks, and scheduled tasks.	Now for structure; paid usage later.
Food search / barcode	Nutritionix API through Edge Functions	Search food items, scan barcodes, parse natural-language food entries, and normalize nutrition data before saving to Supabase.	When Nutrition screen backend begins.
AI meal suggestions	OpenAI API through Edge Functions	Generate meal ideas, macro-friendly suggestions, and smart food guidance without exposing API keys in the app.	When AI feature enters active testing.
In-app subscriptions	RevenueCat	Manage iOS/Android purchases, entitlements, paywalls, receipts, and subscription state.	When premium features are ready to test.
App launch	Apple Developer, Google Play Console, Expo EAS	Build, TestFlight/internal testing, production app submission, and app store releases.	Near beta and store submission.

2. What Is Needed Right Now

Because the UI is done, the immediate paid or account setup should focus on the app backbone only.

- Create or confirm the Supabase project for ONE UP.
- Design the database schema for users, profiles, workouts, progress, nutrition logs, challenges, XP, badges, and subscription entitlements.
- Set up Supabase Auth and Row Level Security before real user data is stored.
- Create the Edge Function structure so Nutritionix, OpenAI, and payment webhooks can be added safely later.
- Decide which features are free vs premium before RevenueCat is wired in.

Not needed immediately: Nutritionix paid usage, OpenAI production spend, RevenueCat live setup, Apple/Google store submission, or monitoring tools. Those should be activated when each feature becomes useful.

3. Subscription Timing Plan

Phase	Subscribe / Set Up	Why It Matters	Hold Off On
Phase 1 - Backend foundation	Supabase project, database, Auth, Edge Functions	Turns the UI into a real app with saved users, secure data, and working app state.	Nutritionix, OpenAI, RevenueCat, store accounts if not yet testing builds.
Phase 2 - Workout and progress data	Supabase only	Workout history, progress charts, scores, streaks, badges, and challenges can be stored and tested without external APIs.	AI meal suggestions and barcode lookup.
Phase 3 - Nutrition feature	Nutritionix API key/plan	Food search, barcode lookup, and nutrition data become real instead of placeholder data.	RevenueCat unless nutrition is premium-gated.
Phase 4 - AI meal suggestions	OpenAI API usage with monthly budget limits	Adds AI meal ideas and smart recommendations, routed safely through Edge Functions.	Expensive models until prompts and usage are proven.
Phase 5 - Premium subscription	RevenueCat, Apple/Google products	Unlocks premium features, tracks entitlements, and supports cross-platform subscriptions.	Full store submission until paywall is tested.
Phase 6 - Beta and launch	Apple Developer, Google Play Console, EAS Build/Submit as needed	Allows TestFlight/internal testing and production app store submission.	Paid monitoring/analytics until beta traffic exists.

4. ONE UP Premium Model Recommendation

The product should feel useful even before payment, then make premium feel like an upgrade to performance, consistency, and personalization.

Free Tier	Premium Tier
Basic workout logging, home dashboard, limited progress summary, basic nutrition log, current level/challenge overview.	AI meal suggestions, advanced progress analytics, recovery map history, custom workout plans, deeper challenge rewards, premium badges, and personalized recommendations.

RevenueCat should be added after the free vs premium boundary is confirmed. That keeps the subscription system tied to actual app value, not guessed features.

5. Cost Control Rules

- Keep API keys out of the mobile app. Store them in Supabase Edge Functions.
- Use development/test plans first; move to paid plans only when the feature is actively being built or tested.
- Set OpenAI budget limits before enabling AI features for testers.
- Log API usage per feature so the client can see what is creating cost.
- Do not connect app store subscriptions until the premium feature list is approved.

6. Client-Facing Recommendation

Since the ONE UP UI is now complete, the next milestone should be backend implementation. The correct next subscription decision is to begin with Supabase because it holds the app together: database, authentication, security rules, and server-side functions.

Nutritionix, OpenAI, RevenueCat, Apple Developer, Google Play Console, and monitoring tools are still important, but they should be activated later, one by one, when the matching feature is ready. This keeps the project financially controlled and prevents paying for services that sit unused for weeks or months.

In short: Supabase now. External APIs and subscriptions later, tied to feature milestones.

7. Reference Notes

These notes are based on official product documentation and pricing pages checked on July 6, 2026. Final pricing should still be confirmed on each provider website at the moment the client subscribes.

Provider	Relevant Official Source	Why It Was Used
Supabase	https://supabase.com/docs/guides/database/overview https://supabase.com/docs/guides/auth https://supabase.com/docs/guides/functions https://supabase.com/pricing	Postgres database, Auth, Row Level Security, Edge Functions, and plan timing.
Nutritionix	https://docx.syndigo.com/developers/docs/natural-language-for-nutrients	Food/nutrition API access pattern and secure server-side key handling.
OpenAI	https://platform.openai.com/docs/pricing https://platform.openai.com/docs/guides/production-best-practices	Usage-based AI pricing, production safety, and cost controls.
RevenueCat	https://www.revenuecat.com/pricing/ https://www.revenuecat.com/docs/getting-started/installation/reactnative	Subscription management, SDKs, entitlement backend, and pricing timing.
Expo	https://docs.expo.dev/submit/introduction/	App build and submission timing for TestFlight/App Store/Play Store.

Prepared for ONE UP. This document is a software planning recommendation, not a commitment to immediate paid subscriptions.